



Hanzo: A Crypto-Commerce Platform for Every Business

Zach Kelling

Hanzo AI

2018

Hanzo: A Crypto-Commerce Platform for Every Business

Zach Kelling

Hanzo AI (Techstars '17, Kansas City)

`zach@hanzo.ai`

2018

Abstract

We describe Hanzo, a unified platform for cryptocurrency payments, smart contract deployment, and digital wallet management targeting businesses with no prior blockchain expertise. The system exposes a single REST API through which merchants accept Bitcoin and Ethereum payments with instant fiat settlement, deploy parameterized smart contracts from an audited template library, and manage both custodial and non-custodial wallets. Two client-side JavaScript libraries—Shop.js for checkout UI components and Coin.js for token operations—reduce frontend integration to fewer than twenty lines of code. Settlement occurs in the merchant’s local currency within one business day at a rate locked at transaction confirmation time, eliminating price-volatility risk entirely. The platform supports ERC-20 fungible tokens [3] and ERC-721 non-fungible tokens [4], with an architecture designed for extension to additional chains and token standards as the ecosystem matures.

1 Introduction

By early 2018, global cryptocurrency market capitalization exceeded \$800 billion and the number of active wallet addresses surpassed 25 million. Yet merchant infrastructure lagged far behind consumer adoption. The typical integration path required a business to: (1) operate or interface with a full blockchain node; (2) manage private keys and transaction signing; (3) handle confirmation latency and chain reorganization risk; (4) convert volatile cryptocurrency proceeds to fiat on an exchange; and (5) implement regulatory compliance checks. Each step introduced operational risk, and the aggregate complexity deterred all but the most technically capable merchants.

Stripe demonstrated that the best payment integration is one where the developer writes as little payments code as possible [6]. Hanzo applies this principle to cryptocurrency. A single API call creates a payment intent; a single webhook confirms settlement. Between those two events, the platform handles address generation, confirmation monitoring, exchange execution, fiat conversion, and bank transfer.

Bitcoin [1] proved that trustless value transfer is possible. Ethereum [2] proved that programmable contracts can automate business logic on-chain. Hanzo provides the commercial infrastructure layer that makes both accessible to any business, regardless of blockchain literacy.

2 Architecture

The platform is organized into three subsystems: a payment gateway, a smart contract engine, and an operations dashboard. All three share a common authentication layer, a unified data model, and a single set of API credentials.

2.1 Payment Gateway

The payment gateway accepts cryptocurrency and settles in fiat. Its responsibilities are:

1. **Address generation.** For each payment intent, the gateway derives a unique deposit address from a hierarchical deterministic (HD) wallet [5]. Address reuse is never permitted.
2. **Rate locking.** At intent creation time, the gateway queries partnered exchange order books and locks a conversion rate for a configurable window (default: 15 minutes). The locked rate includes a spread that covers execution slippage and platform fees.
3. **Confirmation monitoring.** The gateway tracks the deposit address across the relevant blockchain and waits for a configurable number of confirmations (default: 1 for Ethereum, 3 for Bitcoin). For high-value transactions, merchants may require additional confirmations.
4. **Exchange execution.** Upon sufficient confirmations, the gateway executes a market sell on one or more partnered exchanges. The sell is atomic from the merchant’s perspective: if the locked rate cannot be met, the platform absorbs the difference from its liquidity reserves.
5. **Fiat settlement.** Net proceeds are transferred to the merchant’s bank account via ACH (US), SEPA (EU), or SWIFT (international), typically within one business day.

2.2 Smart Contract Engine

The smart contract engine manages the lifecycle of on-chain programs. It stores a library of audited Solidity templates, each accepting a parameter struct via the Hanzo API. The engine compiles, deploys, and verifies the contract on the target chain, then monitors its execution. Merchants interact with smart contracts entirely through REST endpoints; no Solidity toolchain is required.

2.3 Operations Dashboard

The Hanzo Dashboard is a web application that provides real-time visibility into all platform operations. It displays payment status across currencies and geographies, smart contract state and event logs, token sale progress with investor-level detail, and aggregated business metrics. The dashboard exposes the same data available through the API, formatted for non-technical operators.

3 Developer SDKs

The platform provides server-side libraries in Node.js, Python, Go, Ruby, and Haskell, each wrapping the REST API with language-idiomatic types and error handling. Two client-side JavaScript libraries handle frontend integration.

3.1 Shop.js

Shop.js provides pre-built UI components for payment and checkout flows. Components handle input validation, card number formatting, browser capability detection, and responsive layout. A complete checkout form requires importing the library and mounting a single component:

```
import { Checkout } from '@hanzo/shop.js'

Checkout.mount('#payment-form', {
```

```

    apiKey: 'pk_live_...',
    currency: 'usd',
    amount: 4999
  })

```

The library renders a form that accepts both fiat card payments and cryptocurrency, routing each to the appropriate backend flow. Style customization is handled through CSS custom properties.

3.2 Coin.js

Coin.js is a lightweight library for token-specific operations: accepting cryptocurrency payments inline, displaying wallet balances, and interacting with token sale contracts. It wraps the Web3 provider detection and fallback logic that would otherwise require significant boilerplate:

```

import { TokenSale } from '@hanzo/coin.js'

const sale = new TokenSale({
  apiKey: 'pk_live_...',
  contract: 'sale_abc123'
})
sale.mount('#token-purchase')

```

Both libraries share a common event system, allowing developers to compose fiat and crypto payment flows within a single checkout experience.

4 Smart Contract Templates

The template library covers four categories of on-chain business logic. Each template is deployed through the API by supplying a JSON parameter object; the engine handles compilation, deployment, and source verification on Etherscan.

4.1 Token Sales (ICOs)

The ICO template implements a configurable crowdsale contract compliant with the ERC-20 standard [3]. Parameters include:

- Hard cap and soft cap denominated in ETH or USD-equivalent.
- Pricing tiers with time-based or volume-based transitions.
- Whitelist enforcement via Merkle root.
- Vesting schedules for team and advisor allocations.
- Refund mechanism if the soft cap is not met.

The contract is pausable by the contract owner and emits standard ERC-20 **Transfer** events for wallet compatibility.

4.2 Airdrops

The airdrop template distributes tokens to a list of addresses. Rather than executing individual transfers (which costs $O(n)$ gas for n recipients), the contract uses a Merkle-proof-based claiming mechanism. The deployer uploads a recipient list; the engine computes the Merkle root and stores it on-chain. Each recipient submits a proof to claim their allocation, paying their own gas. Total deployment cost is constant regardless of recipient count.

4.3 Collectibles

The collectibles template deploys ERC-721 non-fungible token (NFT) contracts [4]. Each token carries a metadata URI pointing to an off-chain JSON descriptor (name, description, image, attributes). The template supports:

- Sequential or random minting.
- Royalty configuration for secondary sales.
- Marketplace listing hooks for integration with third-party platforms.
- Metadata freezing after a configurable period.

4.4 Referral Programs

The referral template implements on-chain referral tracking. When a referred user completes a qualifying action (purchase, registration, or token acquisition), the contract automatically distributes a configurable reward to the referrer's address. Referral chains of arbitrary depth are supported, with diminishing reward percentages at each level.

5 Wallet Solutions

The platform supports two wallet modes, selectable per-user or per-transaction through a single API parameter.

5.1 Custodial Wallets

In custodial mode, Hanzo manages private keys in a hardware security module (HSM) cluster. Key material never leaves the HSM boundary. Transaction signing occurs server-side, and wallet recovery uses standard identity verification (email, phone, government ID). This mode targets consumer-facing applications where ease of use outweighs the trust assumption.

5.2 Non-Custodial Wallets

In non-custodial mode, keys are generated and stored on the user's device. The Hanzo SDK provides key derivation (BIP-32/BIP-44), mnemonic backup (BIP-39), and transaction construction, but never transmits private key material to Hanzo servers. Signing occurs client-side. This mode targets users who demand full sovereignty over their funds.

Property	Custodial	Non-Custodial
Key storage	HSM	User device
Signing	Server-side	Client-side
Recovery	Identity verification	Mnemonic phrase
Trust model	Hanzo as custodian	Self-custody
API surface	Identical	Identical

Table 1: Wallet mode comparison. Both modes expose the same API; the mode parameter controls key management and signing behavior.

6 Settlement and Foreign Exchange

Settlement is the operation that distinguishes Hanzo from a raw blockchain interface. The merchant receives fiat; Hanzo absorbs all cryptocurrency exposure.

6.1 Rate Locking

At payment intent creation, the gateway samples order book depth across partnered exchanges and computes a guaranteed rate that includes:

1. The mid-market rate at the time of intent creation.
2. A slippage buffer based on the payment amount and current liquidity.
3. The platform fee (configurable per merchant, default 1%).

The locked rate is valid for the confirmation window. If the market moves adversely during confirmation, the platform’s liquidity reserve absorbs the difference. If the market moves favorably, the surplus accrues to the reserve. Over a large volume of transactions, the reserve is self-replenishing.

6.2 FX Conversion

For merchants settling in non-USD currencies, the gateway chains two conversions: crypto-to-USD on the partnered exchange, then USD-to-local via institutional FX rails. The combined rate is quoted at intent creation and locked for the confirmation window, so the merchant sees a single, stable number in their local currency regardless of how many intermediate conversions occur.

6.3 Settlement Rails

Region	Rail	Typical Latency
United States	ACH	T+1
EU / EEA	SEPA	T+1
United Kingdom	Faster Payments	Same day
International	SWIFT	T+2-3

Table 2: Settlement rails by merchant region.

7 Multi-Chain Support

At launch, the platform supports two chains: Bitcoin (UTXO model) and Ethereum (account model, including all ERC-20 and ERC-721 tokens). The architecture is designed for chain-agnostic extension.

Each supported chain is represented by a *chain adapter*—a module that implements a standard interface:

- `generateAddress(hdPath)` — derive a deposit address.
- `watchAddress(addr, callback)` — monitor for incoming transactions.
- `confirmTransaction(txHash, minConfirms)` — wait for finality.
- `broadcastTransaction(signedTx)` — submit a signed transaction.

Adding a new chain requires implementing this interface and registering the adapter with the gateway. The payment API, settlement logic, and merchant dashboard are chain-agnostic; they operate on the abstract transaction model produced by the adapter.

At the time of writing, adapter development is underway for Litecoin, Bitcoin Cash, and EOS. The ERC-20 adapter automatically supports any token deployed on the Ethereum mainnet without per-token configuration.

8 Deployment

Hanzo is a managed platform. Merchants do not operate infrastructure. The production deployment consists of:

- **API servers.** Stateless HTTP services behind a load balancer, horizontally scaled by request volume.
- **Chain watchers.** One process per supported chain, connected to geographically distributed full nodes for redundancy. Watchers are single-writer to a shared transaction log, ensuring exactly-once processing.
- **Exchange connectors.** Persistent WebSocket connections to partnered exchange APIs for real-time order book data and trade execution.
- **HSM cluster.** Dedicated hardware security modules for custodial key storage, deployed in a quorum configuration requiring k of n modules to sign.
- **Settlement engine.** A batch processor that aggregates confirmed transactions, computes net settlement amounts per merchant, and initiates bank transfers on a daily cycle.
- **Dashboard.** A single-page application served from a CDN, consuming the same API available to developers.

All components emit structured logs and metrics to a centralized observability stack. Uptime is monitored at `status.hanzo.ai` with public incident reporting.

8.1 API Versioning

The API is versioned with a date-based scheme. Breaking changes are released under a new version header; older versions continue to function for a minimum of twelve months. Non-breaking additions (new fields, new endpoints) are available immediately on all versions. This policy ensures that merchants upgrade on their own schedule without surprises.

9 Security

- **Smart contract audits.** Every template undergoes third-party audit before inclusion in the library. Audit reports are published and linked from the API documentation.
- **Fraud detection.** Transaction pattern analysis flags anomalous behavior (unusual amounts, rapid-fire transactions, known tainted addresses) and holds the transaction for manual review.
- **Key security.** Custodial keys are stored exclusively in HSMs (FIPS 140-2 Level 3). Non-custodial keys never leave the client device.
- **Transport security.** All API traffic is TLS 1.2+ with certificate pinning available in mobile SDKs.
- **Compliance.** KYC/AML checks are integrated at the account level, with jurisdiction-specific rule sets configurable per merchant.

10 Conclusion

Hanzo reduces cryptocurrency commerce to an API integration. A merchant adds a few lines of JavaScript, receives fiat in their bank account, and gains access to smart contract-powered business models—token sales, airdrops, collectibles, referral programs—without writing Solidity or operating blockchain infrastructure. The zero-volatility settlement model eliminates the primary financial risk of accepting cryptocurrency, and the dual custodial/non-custodial wallet architecture lets businesses serve both mainstream consumers and self-sovereign users through a single integration.

The platform is operational and accepting merchants. Documentation is available at `docs.hanzo.ai`; developer support is available at `#hanzo` on Freenode.

References

- [1] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [2] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform. 2014.
- [3] F. Vogelsteller and V. Buterin. ERC-20 token standard. Ethereum Improvement Proposal 20, 2015.
- [4] W. Entriken, D. Shirley, J. Evans, and N. Sachs. ERC-721 non-fungible token standard. Ethereum Improvement Proposal 721, 2018.
- [5] P. Wuille. BIP-32: Hierarchical deterministic wallets. Bitcoin Improvement Proposal 32, 2012.
- [6] Stripe, Inc. Stripe payments documentation. <https://stripe.com/docs>, 2018.